

Formation Engine Methodology

Section 10b — Defect Microstate Classification: Third Regime for Occupancy and Disorder

Formation Engine Development Team

Version 3.0.0 — Unified Clustering Architecture

1 Introduction: Beyond Perfect Crystals

Sections 10–13 established crystal structure generation, supercell construction, and perfect lattice relaxation. Real materials, however, rarely occupy a single ideal configuration. Experimental specimens exhibit:

- **Vacancies and interstitials** (point defects)
- **Substitutional disorder** (dopants, solid solutions)
- **Antisite defects** (wrong atom on wrong sublattice)
- **Charge compensation patterns** (oxygen vacancies $\leftrightarrow$ cation oxidation states)
- **Supercell-scale ordering motifs** (short-range order, clustering)

These features define a **continuum of microstates** within a parent phase, not discrete polymorphs. This section introduces the **defect (microstate) regime**—a third classification paradigm complementing polymorphs (§10: same chemistry, different structure) and isomorphs (§10: same structure, different chemistry) with a unified deterministic pipeline.

2 Regime Definition and Invariants

2.1 Mathematical Formulation

A structure instance is represented as:

$$\mathcal{S} = (\mathbf{L}, \{\mathbf{r}_i\}, \{s_i\}) \tag{1}$$

where:

- $\mathbf{L}$ is the lattice matrix (3×3 , columns are $\{\mathbf{a}, \mathbf{b}, \mathbf{c}\}$)
- $\mathbf{r}_i$ are fractional coordinates of atom i
- s_i is the species label (atomic number or type identifier)

Definition 1 (Defect Microstate Regime). *The defect microstate regime classifies structures by:*

$$\begin{aligned} \textbf{Invariant:} \quad & \text{parent prototype } \Pi \text{ (topology/space group)} \\ \textbf{Variable:} \quad & \text{occupancy configuration } \Omega \text{ (site decoration + vacancies)} \\ & \text{local distortion field } \Delta \text{ (geometry perturbations)} \end{aligned} \tag{2}$$

Formally, let $\Pi(\mathcal{S})$ denote the prototype assignment (via topology fingerprint). The defect class of $\mathcal{S}$ is:

$$\text{DefectClass}(\mathcal{S}) = \text{Cluster}_\epsilon(\text{FP}_{\text{def}}(\mathcal{S}) \mid \Pi(\mathcal{S}) = \Pi_0) \tag{3}$$

where clustering is performed within a fixed parent prototype Π_0 .

2.2 Regime Comparison

Table ?? contrasts the three classification regimes:

Table 1: Unified Classification Regimes			
Regime	Invariant	Variable	Output
Polymorph	Composition	Structure	α, β, γ phases
Isomorph	Prototype	Chemistry	Decoration variants
Defect Microstate	Parent phase	Occupancy + disorder	$\omega_1, \omega_2, \dots$

Key insight: All three regimes use the *same clustering engine* with different fingerprints. This unification enables:

- One canonicalization routine
- One neighbor-building algorithm
- One deterministic clustering implementation
- Two fingerprint constructors: FP_{proto} (structure) and FP_{deco} (chemistry/defects)

3 Deterministic Pipeline

3.1 Core Algorithm

The defect classification pipeline (Algorithm ??) extends the crystal supercell workflow (§10) with microstate-aware fingerprints:

Algorithm 1 Defect Microstate Classification

Require: Set of relaxed structures $\{\mathcal{S}_1, \dots, \mathcal{S}_N\}$

Ensure: Defect class assignments $\{\omega_1, \dots, \omega_N\}$

```
1: Phase 1: Parent Prototype Assignment
2: for each  $\mathcal{S}_i$  do
3:    $\mathcal{S}_i \leftarrow \text{canonicalize}(\mathcal{S}_i)$  ▷ Wrap coords, order atoms
4:    $\mathbf{G}_i \leftarrow \text{build\_neighbors}(\mathcal{S}_i)$  ▷ Fixed cutoff rule
5:    $\text{FP}_{\text{proto}}^{(i)} \leftarrow \text{proto\_fingerprint}(\mathbf{G}_i, \mathcal{S}_i)$  ▷ Topology hash + CN
6: end for
7:  $\{\Pi_1, \dots, \Pi_M\} \leftarrow \text{cluster}(\{\text{FP}_{\text{proto}}^{(i)}\}, \epsilon_{\text{proto}})$  ▷ Prototype IDs
8:
9: Phase 2: Defect Microstate Clustering (within each  $\Pi_k$ )
10: for each prototype  $\Pi_k$  do
11:    $\mathcal{I}_k \leftarrow \{i : \Pi(\mathcal{S}_i) = \Pi_k\}$  ▷ Structures in parent  $\Pi_k$ 
12:   for each  $i \in \mathcal{I}_k$  do
13:      $\text{FP}_{\text{def}}^{(i)} \leftarrow \text{defect\_fingerprint}(\mathcal{S}_i, \Pi_k)$  ▷ Eq. (??)
14:   end for
15:    $\{\omega_{k,1}, \omega_{k,2}, \dots\} \leftarrow \text{cluster}(\{\text{FP}_{\text{def}}^{(i)}\}_{i \in \mathcal{I}_k}, \epsilon_{\text{def}})$ 
16: end for
17: return  $\{(\Pi(\mathcal{S}_i), \omega(\mathcal{S}_i)) : i = 1, \dots, N\}$ 
```

3.2 Fingerprint Construction

3.2.1 Prototype Fingerprint (Chemistry-Agnostic)

The prototype fingerprint captures *topology* and *geometry* independent of specific species:

$$\text{FP}_{\text{proto}}(\mathcal{S}) = \left(\text{WL_hash}(\mathbf{G}), \mathbf{h}_{\text{CN}}, \mathbf{h}_{\text{RDF}} \right) \quad (4)$$

where:

- $\text{WL_hash}(\mathbf{G})$ is the Weisfeiler-Lehman graph hash (canonical topology signature)
- $\mathbf{h}_{\text{CN}}$ is the coordination number histogram (per site class, if known)
- $\mathbf{h}_{\text{RDF}}$ is a normalized radial distribution function (optional, for geometric refinement)

All components are computed deterministically from $\mathbf{G}$ (the neighbor graph at fixed cutoff) and canonicalized coordinates.

3.2.2 Defect Fingerprint (Chemistry + Occupancy)

The defect fingerprint captures *local disorder* within a fixed parent:

$$\text{FP}_{\text{def}}(\mathcal{S}) = \left(\mathbf{o}, \mathbf{v}, \mathbf{m}, \mathbf{d} \right) \quad (5)$$

where:

$\mathbf{o}$ = site-class occupancy vector	(counts or fractions per Wyckoff position)
$\mathbf{v}$ = vacancy/interstitial vector	(deviation from ideal site count)
$\mathbf{m}$ = substitution/antisite signature	(sublattice decoration pattern)
$\mathbf{d}$ = local distortion signature	(CN deviation + bond-length residual histograms)

Component definitions:

1. **Occupancy vector $\mathbf{o}$:** For each site class $\alpha \in \{\alpha_1, \dots, \alpha_K\}$ (determined from parent prototype via symmetry or neighbor topology):

$$o_\alpha = \frac{N_\alpha}{\langle N_\alpha \rangle_{\text{ideal}}} \quad (6)$$

where N_α is the actual count and $\langle N_\alpha \rangle_{\text{ideal}}$ is the expected count from the parent prototype.

2. **Vacancy vector $\mathbf{v}$:**

$$v_\alpha = 1 - o_\alpha \quad (\text{negative indicates interstitial region}) \quad (7)$$

3. **Substitution signature $\mathbf{m}$:** Species composition per site class, expressed as a histogram over atomic numbers (or type IDs). For site class α :

$$m_{\alpha,Z} = \frac{\#\{\text{atoms of species } Z \text{ on sites of class } \alpha\}}{N_\alpha} \quad (8)$$

Antisite defects (A on B sublattice) manifest as nonzero $m_{\beta,A}$ where β is expected to host species B.

4. **Distortion signature $\mathbf{d}$:** For each atom i , compute:

$$\text{CN}_i = \text{coordination number from neighbor graph} \quad (9)$$

$$\langle r_i \rangle = \text{mean nearest-neighbor distance} \quad (10)$$

Then build histograms:

$$\mathbf{d} = (\mathbf{h}_{\text{CN-dev}}, \mathbf{h}_{r\text{-dev}}) \quad (11)$$

where:

$$\mathbf{h}_{\text{CN-dev}} : \text{histogram of } (\text{CN}_i - \langle \text{CN} \rangle_{\text{ideal}}) \quad (12)$$

$$\mathbf{h}_{r\text{-dev}} : \text{histogram of } (\langle r_i \rangle - \langle r \rangle_{\text{ideal}}) \quad (13)$$

3.3 Distance Metric and Clustering

Define a weighted Euclidean distance over defect fingerprints:

$$D_{\text{def}}(\mathcal{S}_a, \mathcal{S}_b) = w_o \|\mathbf{o}_a - \mathbf{o}_b\| + w_v \|\mathbf{v}_a - \mathbf{v}_b\| + w_m \delta(\mathbf{m}_a, \mathbf{m}_b) + w_d \text{dist}(\mathbf{d}_a, \mathbf{d}_b) \quad (14)$$

where:

- $\|\cdot\|$ is the ℓ^2 norm for continuous vectors

- $\delta(\mathbf{m}_a, \mathbf{m}_b)$ is a discrete substitution distance (e.g., sum of absolute differences of histograms)
- $\text{dist}(\mathbf{d}_a, \mathbf{d}_b)$ is the Wasserstein distance or χ^2 statistic between histograms
- (w_o, w_v, w_m, w_d) are fixed weights (default: uniform)

Clustering proceeds deterministically via threshold assignment:

$$\text{Cluster}_\epsilon(\{\text{FP}_1, \dots, \text{FP}_N\}) = \{\omega_1, \dots, \omega_N\} \quad (15)$$

where:

1. Process structures in canonical order (stable hash of FP_{proto})
2. For each structure i :
 - Compute $D_{\text{def}}(\text{FP}_i, \text{centroid}_k)$ for all existing clusters k
 - If $\min_k D_{\text{def}}(\text{FP}_i, \text{centroid}_k) < \epsilon$, assign to that cluster
 - Otherwise, create new cluster with ID $\text{hash}(\text{FP}_i)$
3. Update centroids as running means

This avoids stochasticity from seeding or iteration order. The cluster ID is a deterministic hash, not a sequential integer.

4 Labeling and Statistics

4.1 Symbolic Labels (Optional)

Defect class labels are *derived properties*, not inputs. If symbolic labels (e.g., $\omega_1, \omega_2, \omega_3$) are desired, they are assigned deterministically by:

$$\text{Rank clusters by: } \begin{cases} \text{Mean energy per formula unit (if available)} \\ \text{Population (number of structures in cluster)} \\ \text{Lexicographic hash of centroid fingerprint} \end{cases} \quad (16)$$

Labels are then assigned $\omega_1, \omega_2, \dots$ in rank order. This ensures reproducibility: the same dataset always yields the same labels.

4.2 Per-Class Statistics

For each defect class ω_k within parent Π , compute:

$$\langle E \rangle_{\omega_k} = \frac{1}{N_k} \sum_{i \in \omega_k} E_i \quad (\text{mean energy per formula unit}) \quad (17)$$

$$\sigma_E^2(\omega_k) = \frac{1}{N_k} \sum_{i \in \omega_k} (E_i - \langle E \rangle)^2 \quad (\text{energy variance}) \quad (18)$$

$$f_{\omega_k} = \frac{N_k}{N_{\text{total}}} \quad (\text{population fraction within } \Pi) \quad (19)$$

$$\langle \mathbf{o} \rangle_{\omega_k} = \frac{1}{N_k} \sum_{i \in \omega_k} \mathbf{o}_i \quad (\text{mean occupancy vector}) \quad (20)$$

$$\langle \mathbf{v} \rangle_{\omega_k} = \frac{1}{N_k} \sum_{i \in \omega_k} \mathbf{v}_i \quad (\text{mean vacancy fraction}) \quad (21)$$

These statistics enable:

- **Thermodynamic interpretation:** Classes with lower $\langle E \rangle$ and higher f are thermodynamically favored under the simulation conditions
- **Defect concentration analysis:** $\langle v \rangle$ quantifies typical vacancy/interstitial densities
- **Stability screening:** Classes with low σ_E^2 are structurally consistent; high σ_E^2 suggests heterogeneous microstates

5 Unified Implementation

5.1 Shared Core Functions

All three regimes (polymorphs, isomorphs, defects) share a common implementation:

Listing 1: Unified Clustering Core (C++20)

```
namespace atomistic {
namespace classify {

// Canonicalize structure: wrap coords to [0,1), order atoms
Structure canonicalize(const Structure& s);

// Build neighbor graph from fixed cutoff rule
NeighborGraph build_neighbors(const Structure& s, double cutoff);

// Fingerprints
struct ProtoFingerprint {
    uint64_t topology_hash;    // Weisfeiler-Lehman hash
    std::vector<int> CN_hist;  // Coordination number histogram
    std::vector<double> RDF;   // Radial distribution function
};

struct DefectFingerprint {
    std::vector<double> occupancy;    // o vector
    std::vector<double> vacancy;     // v vector
    std::map<int, std::vector<double>> substitution; // m (Z -> histogram)
    std::vector<double> CN_deviation; // d: CN histogram
    std::vector<double> bond_deviation; // d: bond length histogram
};

ProtoFingerprint compute_proto_fp(const NeighborGraph& g, const Structure& s);
DefectFingerprint compute_defect_fp(const Structure& s, const Prototype& parent);

// Distance functions
double distance(const ProtoFingerprint& a, const ProtoFingerprint& b);
double distance(const DefectFingerprint& a, const DefectFingerprint& b,
               const std::array<double, 4>& weights = {1,1,1,1});

// Clustering (deterministic threshold assignment)
template<typename FP>
std::vector<uint64_t> cluster(const std::vector<FP>& fingerprints,
                           double epsilon);
```

```

} // namespace classify
} // namespace atomistic

```

5.2 Module A: Polymorph Detection

Listing 2: Polymorph Classifier (Same Chemistry $\rightarrow$ Structure Phases)

```

std::map<std::string, std::vector<uint64_t>>
classify_polymorphs(const std::vector<Structure>& structures) {
    // Group by reduced formula
    auto grouped = group_by_formula(structures);

    std::map<std::string, std::vector<uint64_t>> results;

    for (auto& [formula, structs] : grouped) {
        std::vector<ProtoFingerprint> fps;
        for (auto& s : structs) {
            auto canon = canonicalize(s);
            auto graph = build_neighbors(canon, FIXED_CUTOFF);
            fps.push_back(compute_proto_fp(graph, canon));
        }

        // Cluster on prototype fingerprint (ignore chemistry)
        auto clusters = cluster(fps, PROTO_EPSILON);

        // Rank by energy assign , , labels
        auto labels = assign_greek_labels(clusters, structs);

        results[formula] = labels;
    }

    return results;
}

```

5.3 Module B: Isomorph Detection

Listing 3: Isomorph Classifier (Same Structure $\rightarrow$ Chemistry Variants)

```

std::map<uint64_t, std::vector<uint64_t>>
classify_isomorphs(const std::vector<Structure>& structures) {
    std::map<uint64_t, std::vector<uint64_t>> results;

    // Phase 1: Assign prototype IDs
    std::vector<ProtoFingerprint> proto_fps;
    for (auto& s : structures) {
        auto canon = canonicalize(s);
        auto graph = build_neighbors(canon, FIXED_CUTOFF);
        proto_fps.push_back(compute_proto_fp(graph, canon));
    }

    auto proto_clusters = cluster(proto_fps, PROTO_EPSILON);
}

```

```

// Phase 2: Within each prototype, cluster by chemistry (decoration)
for (uint64_t proto_id : unique(proto_clusters)) {
    auto indices = where(proto_clusters, proto_id);

    std::vector<DefectFingerprint> deco_fps;
    for (size_t i : indices) {
        deco_fps.push_back(compute_defect_fp(structures[i], proto_id));
    }

    auto variant_clusters = cluster(deco_fps, DECO_EPSILON);
    results[proto_id] = variant_clusters;
}

return results;
}

```

5.4 Module C: Defect Microstate Detection

Listing 4: Defect Classifier (Same Parent $\rightarrow$ Occupancy/Disorder Classes)

```

std::map<uint64_t, std::vector<uint64_t>>
classify_defects(const std::vector<Structure>& structures) {
    // Phase 1: Assign parent prototype IDs (same as isomorph Phase 1)
    auto proto_clusters = assign_prototypes(structures, PROTO_EPSILON);

    std::map<uint64_t, std::vector<uint64_t>> results;

    // Phase 2: Within each parent, cluster by defect fingerprint
    for (uint64_t parent_id : unique(proto_clusters)) {
        auto indices = where(proto_clusters, parent_id);

        std::vector<DefectFingerprint> defect_fps;
        for (size_t i : indices) {
            defect_fps.push_back(compute_defect_fp(structures[i], parent_id));
        }

        // Weights: emphasize vacancy + substitution over pure distortion
        std::array<double, 4> weights = {1.0, 1.5, 2.0, 0.5}; // o, v, m, d

        auto defect_clusters = cluster(defect_fps, DEFECT_EPSILON, weights);

        // Rank by population or mean energy assign , , labels
        auto labels = assign_omega_labels(defect_clusters, structures[indices]);

        results[parent_id] = labels;
    }

    return results;
}

```

6 Example: MgO with Vacancy Disorder

6.1 System Setup

Consider a simulation generating 500 relaxed MgO structures at 1200 K with:

- 90% stoichiometric MgO (rocksalt structure, $Fm\bar{3}m$)
- 8% structures with $\sim 1\%$ Mg vacancies
- 2% structures with O vacancies (charge-compensated by Mg^+)

6.2 Classification Pipeline

Step 1: Prototype Assignment. All 500 structures cluster to **one prototype ID** (rocksalt FCC sublattice topology with octahedral/tetrahedral voids).

Step 2: Defect Fingerprinting. For each structure, compute FP_{def} :

- $\mathbf{o} = (o_{\text{Mg}}, o_{\text{O}})$ where $o_{\text{Mg}} \approx 0.99$ for vacancy-containing structures
- $\mathbf{v} = (1 - o_{\text{Mg}}, 1 - o_{\text{O}})$ captures vacancy fractions
- $\mathbf{m}$ captures whether Mg or O sites host vacancies (antisite-like)
- $\mathbf{d}$ shows distortion histograms (CN deviations near vacancies)

Step 3: Clustering. With $\epsilon_{\text{def}} = 0.05$, clustering yields:

- ω_1 : 450 structures (stoichiometric, CN=6 everywhere)
- ω_2 : 40 structures (Mg vacancies, $\langle v_{\text{Mg}} \rangle \approx 0.01$)
- ω_3 : 10 structures (O vacancies, $\langle v_{\text{O}} \rangle \approx 0.01$)

Step 4: Statistics.

$$\langle E \rangle_{\omega_1} = -10.3 \text{ eV/fu} \quad (\text{baseline stoichiometric}) \quad (22)$$

$$\langle E \rangle_{\omega_2} = -10.1 \text{ eV/fu} \quad (+0.2 \text{ eV penalty for Mg vacancy}) \quad (23)$$

$$\langle E \rangle_{\omega_3} = -9.8 \text{ eV/fu} \quad (+0.5 \text{ eV penalty for O vacancy}) \quad (24)$$

Population fractions:

$$f_{\omega_1} = 0.90, \quad f_{\omega_2} = 0.08, \quad f_{\omega_3} = 0.02 \quad (25)$$

This automatically recovers the defect chemistry from simulation output, with no manual classification or pre-specified labels.

7 Validation and Robustness

7.1 Reproducibility

Determinism: All fingerprints, distances, and cluster assignments are deterministic. Running the pipeline twice on the same dataset produces identical ω labels, even if structure order changes (because canonicalization sorts atoms and cluster IDs are hashes).

Threshold sensitivity: Vary ϵ_{def} from 0.01 to 0.10 and verify:

- ϵ too small $\rightarrow$ over-clustering (every structure is its own class)
- ϵ too large $\rightarrow$ under-clustering (all structures merge to one class)
- Optimal ϵ yields stable cluster count across small perturbations

Use the elbow method or silhouette score to auto-tune ϵ (while maintaining determinism by fixing random seeds to zero or using grid search with fixed order).

7.2 Physical Consistency

Energy ordering: For a correct force field and sufficient sampling:

$$\langle E \rangle_{\omega_1} < \langle E \rangle_{\omega_2} < \langle E \rangle_{\omega_3} \quad \Leftrightarrow \quad f_{\omega_1} > f_{\omega_2} > f_{\omega_3} \quad (26)$$

This is a **necessary condition** for thermodynamic validity: lower-energy defect classes should appear more frequently in equilibrium sampling.

Vacancy concentration bounds: For point-defect-dominated regimes:

$$\langle v_\alpha \rangle_{\omega_k} < 0.10 \quad (\text{typically}) \quad (27)$$

Classes with $\langle v \rangle > 0.20$ likely represent amorphous regions or phase transitions, not simple defect microstates.

7.3 Comparison to Manual Classification

Run the pipeline on a known dataset with expert-assigned labels (e.g., from DFT calculations with annotated defect types). Compute:

$$\text{Purity} = \frac{1}{K} \sum_{k=1}^K \frac{\max_{\ell} |\omega_k \cap L_{\ell}|}{|\omega_k|} \quad (28)$$

where L_{ℓ} are the expert labels. Purity > 0.90 indicates strong agreement.

8 Limitations and Honest Assessment

Site-class determination. The pipeline assumes site classes (Wyckoff positions or sublattices) are either:

1. Known from the parent prototype (via space group)
2. Inferred from neighbor topology (via graph partitioning)

For highly disordered or amorphous regions, site-class assignment may be ambiguous. In such cases, fall back to global composition histograms (no per-site resolution).

Large supercells. For supercells with $N > 1000$ atoms, fingerprint computation scales as $\mathcal{O}(N^2)$ for neighbor graphs. Optimization strategies:

- Use cell lists or Verlet lists for $\mathcal{O}(N)$ neighbor building
- Subsample atoms for defect fingerprints (e.g., every 10th atom for **d**)
- Cache topology hashes to avoid recomputation

Force field dependence. Energy-based ranking ($\langle E \rangle_{\omega_k}$) depends on force field accuracy. For classical potentials (LJ, Buckingham), expect qualitative correctness (relative energy ordering) but not quantitative precision (absolute formation energies).

Non-equilibrium sampling. If structures are quenched from high temperature or kinetically trapped, population fractions f_{ω_k} do *not* reflect equilibrium thermodynamics. The classification is still valid (microstates are correctly grouped), but energy-vs-population correlations require equilibration.

9 Implementation Status

- **Core fingerprints:** Implemented in `atomistic/classify` (prototype + defect fingerprints)
- **Clustering:** Deterministic threshold assignment in `atomistic/classify/cluster.cpp`
- **Polymorph module:** Tested on 50-structure H₂O dataset (reproduces known ice phases)
- **Isomorph module:** Prototype, awaiting multi-chemistry validation
- **Defect module:** Interface designed, awaiting integration with `atomistic/crystal` supercell builder

10 Conclusion: One Clustering Engine, Three Front-Ends

This section extends the crystal classification framework (§10) to handle realistic disorder. The key contributions are:

1. **Unified clustering architecture:** Three regimes (polymorphs, isomorphs, defects) share the same deterministic pipeline with different fingerprints
2. **Defect fingerprint:** Four-component vector (o, v, m, d) capturing occupancy, vacancies, substitutions, and distortion
3. **Parent-aware clustering:** Defect classification is performed *within* a fixed prototype, avoiding conflation of different structural basins
4. **Deterministic labeling:** Symbolic labels $(\omega_1, \omega_2, \dots)$ are derived from energy/population ranking, not manual annotation
5. **Validation protocol:** Reproducibility, energy ordering, and expert-label purity metrics ensure physical consistency

Transition to production: With polymorphs, isomorphs, and defects unified under one clustering engine, the next step is **automated phase diagram construction** (§12): given a composition range and temperature/pressure grid, iteratively generate structures, classify them, and map the resulting phase space.